

Proven Software with *Executing on* **Low Cost High Integrity Platform**



Thierry Lecomte
October 20, 2016
Sherbrooke

Introduction

- Safety critical systems & standards

Proven software with B

- B (101), Proven with B, Software with B
- Improved “improved development cycle”

Low Cost High Integrity Platform

- Double processor, double toolchain, double software instances
- LCHIP

Introduction

Safety Critical Systems

Systems where life is at risk



Should either work properly or avoid to kill people in case of failure

- Default-free is not from this earth
- Safety case as demonstration

Safety Critical Systems

Standards

• Railways	EN5012{6, 8, 9}	SIL	0, 1, 2, 3, 4	Highest level
• Aeronautics	DO-178C	DAL	E, D, C, B, A	
• Automotive	ISO 26262	ASIL	0, 1, 2, 3, 4	
• Industry	IEC 61508	SIL	0, 1, 2, 3, 4	

SIL3: 10^{-7} failure / hour

SIL4: 10^{-9} failure / hour

μprocessor: 10^{-5} failure / hour



SIL3, SIL4: 2 μprocessors (or more) required
(2oo2, 2oo3, voter, processor+coprocessor, etc.)

Safety Critical Systems

For highest SIL, formal methods are highly recommended ... as well as other means

We are far from

“It compiles hence it works !”

IEC 61508: Software design and dev. (table A.2)

Technique/Measure	Ref	SIL1	SIL2	SIL3	SIL4
1 Fault detection and diagnosis	C.3.1	---	R	HR	HR
2 Error detecting and correcting codes	C.3.2	R	R	R	HR
3a Failure assertion programming	C.3.3	R	R	R	HR
3b Safety bag techniques	C.3.4	---	R	R	R
3c Diverse programming	C.3.5	R	R	R	HR
3d Recovery block	C.3.6	R	R	R	R
3e Backward recovery	C.3.7	R	R	R	R
3f Forward recovery	C.3.8	R	R	R	R
3g Re-try fault recovery mechanisms	C.3.9	R	R	R	HR
3h Memorising executed cases	C.3.10	---	R	R	HR
4 Graceful degradation	C.3.11	R	R	HR	HR
5 Artificial intelligence - fault correction	C.3.12	---	NR	NR	NR
6 Dynamic reconfiguration	C.3.13	---	NR	NR	NR
7a Structured methods including for example, ISD, MASCOT, SADT and Yourdon	C.2.1	HR	HR	HR	HR
7b Semi-formal methods	Table B.7	R	R	HR	HR
7c Formal methods including for example, CCS, CSP, HOL, LOTOS, OBJ, temporal logic, VDM and Z	C.2.4	---	R	R	HR
8 Computer-aided specification tools	B.2.4	R	R	HR	HR

a) Appropriate techniques/measures shall be selected according to the safety integrity level. Alternate or equivalent techniques/measures are indicated by a letter following the number. Only one of the alternate or equivalent techniques/measures has to be satisfied.

b) The measures in this table concerning fault tolerance (control of failures) should be considered with the requirements for architecture and control of failures for the hardware of the programmable electronics in part 2 of this standard.

Safety Critical Systems

Ability to command outputs

- Bijection between processor memory and actuator physical state
- Checked often
- Restrictive mode when check fails (reboot, shutdown, etc.)

Restrictive/degraded mode

- No universal RM, depends on the system
- Trains: usually reduce kinetic energy (emergency breaking)
- Planes: give commands back to human pilot

Proven Software with B & Contribution to Safety Critical Systems

Atelier B initially developed by Fernando Mejia

◆ Development funded for Driverless metro in Paris (L14)

◆ First customer: University of Sherbrooke

◆ METEOR released

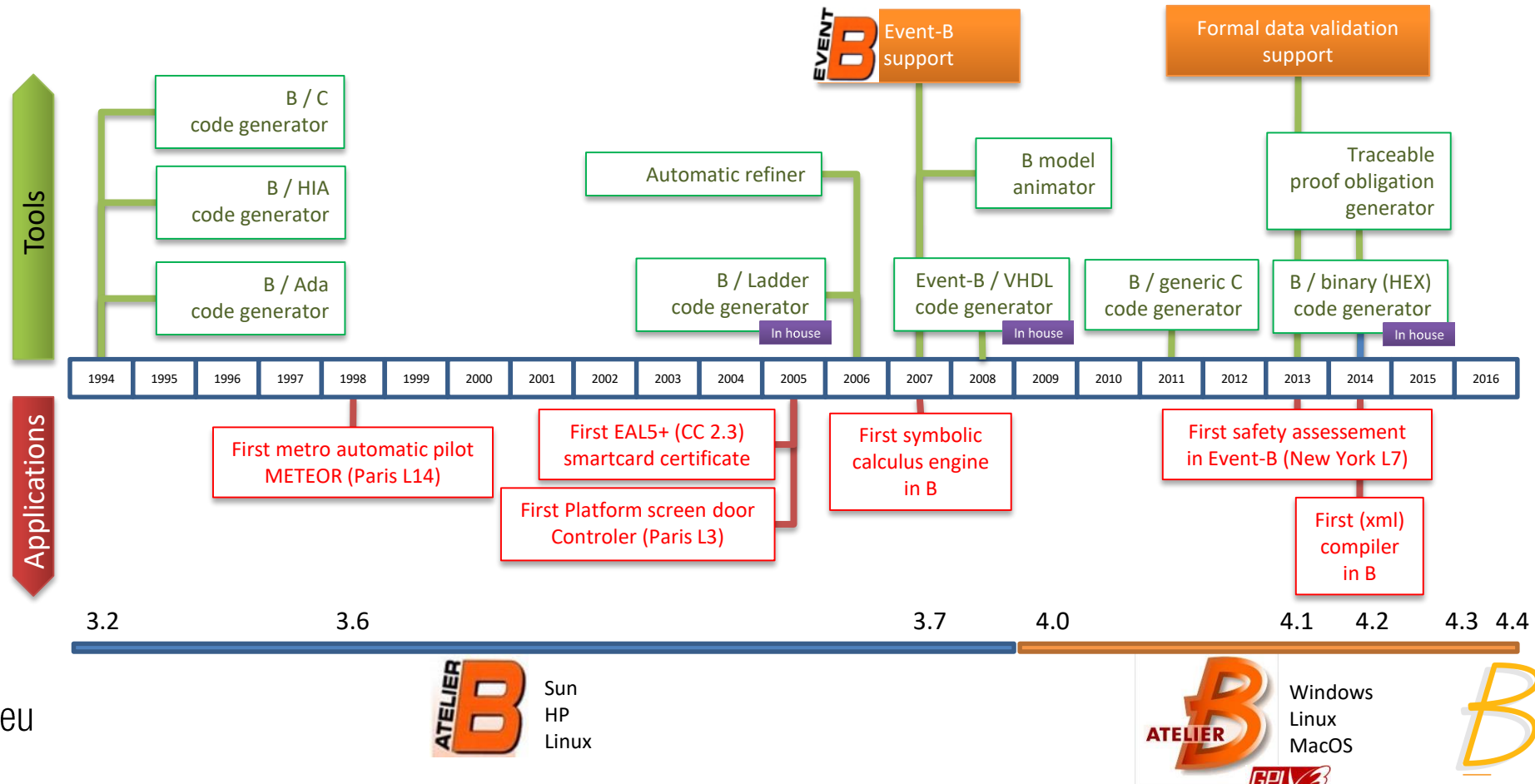
◆ Creation of ClearSy (full ownership)

◆ Atelier B Community + Maintenance

GEC ALSTHOM



MATRA



Disclaimer

- Demos are made with not yet public Atelier B versions (4.4 & 4.5)
- Current version 4.3 can be downloaded at www.atelierb.eu

- formal method (semantics)
 - text-based models of unthreaded programs
 - same language to specify **(MACHINE)** and design **(IMPLEMENTATION)**
 - set theory and first order logic
 - Specification for the “what”
 - Design for the “how” and the architecture

```
MACHINE
  M0
VARIABLES
  xx, ii
INVARIANT
  xx: BOOL &
  ii : INTEGER &
  (ii > 0 => xx = TRUE)
INITIALISATION
  ii := 0 ||
  xx := TRUE
OPERATIONS
  update =
  BEGIN
    xx := TRUE ||
    ii :: 1..3
  END
END
```

B (101)

Variables

- Can be abstract (not implemented), just for reasoning
- Can be concrete: need to be implemented in one and only one implementation

Invariant

- Types for variables (set, function, scalar, boolean, integer, table, etc.)
- Constraints for variables
- Invariant is true all the time
 - ensured by initialisation
 - If true before executing an operation, still true after

Initialisation

- First value for variables
- Can be non-deterministic
- All variables initialized at the same time

Operation

- Modify modelling variables
- A precondition may specify under which conditions the operation is callable
- Specified as a transfer function (no algorithm)

```
MACHINE
  M0
  VARIABLES
    xx, ii
  INVARIANT
    xx: BOOL &
    ii : INTEGER &
    (ii > 0 => xx = TRUE)
  INITIALISATION
    ii := 0 ||
    xx := TRUE
  OPERATIONS
    update =
      BEGIN
        xx := TRUE ||
        ii :: 1..3
      END
  END
```

B (101)

- with proof (models are validated by proof)
 - coherency / compliancy expressed as proof obligations
 - proof obligations are not an option
 - automatic generation
 - Pro: proof equivalent to exhaustive testing
 - Con: proof is semi-automatic (require expert transactions to complete)
 - Proof extent can be checked at a glance

PRJ002 (OK|OK|2|0|100%)

Classical view

Component	TypeChecked	POs Generated	Proof Obligations	Proved	Unproved	B0 Checked
M0	OK	OK	2	2	0	-

What is to be proved ?

What is proved ?

```
MACHINE
  M0
VARIABLES
  xx, ii
INVARIANT
  xx: BOOL &
  ii : INTEGER &
  (ii > 0 => xx = TRUE)
INITIALISATION
  ii := 0 ||
  xx := TRUE
OPERATIONS
  update =
  BEGIN
    xx := TRUE ||
    ii :: 1..3
  END
END
```

=>

```
"Invariant is preserved" &
"Check invariant ((ii) : (INTEGER))"

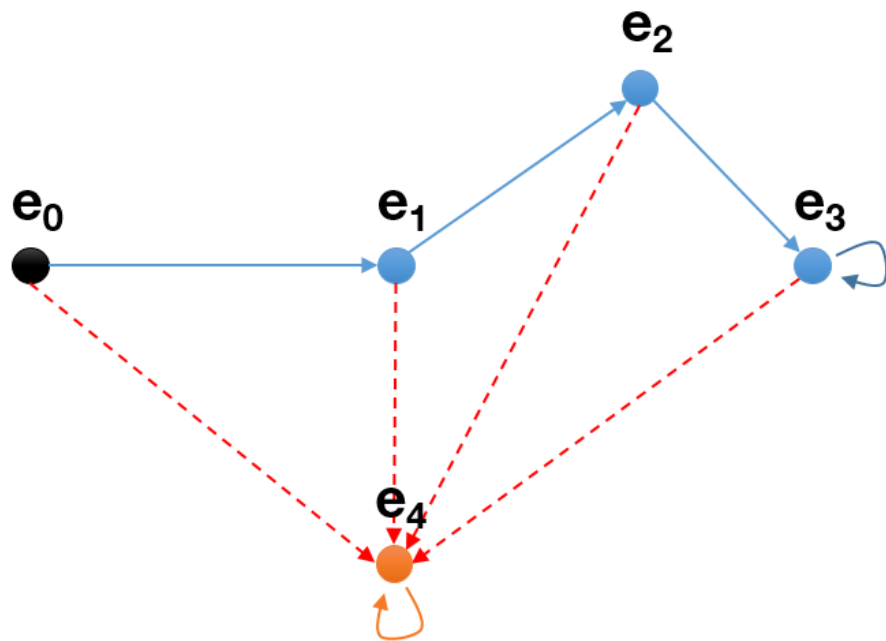
0: INTEGER
```

=>

```
ii$1: 1..3 &
"Invariant is preserved" &
"Check invariant ((ii) : (INTEGER))"

ii$1: INTEGER
```

Example: implementing a state machine



MACHINE

CTX

SETS

STATES = {e0, e1, e2, e3, e4}

CONSTANTS

next

PROPERTIES

```
next = {
    e0 |-> e1,
    e1 |-> e2,
    e2 |-> e3,

    e3 |-> e3,

    e0 |-> e4,
    e1 |-> e4,
    e2 |-> e4,
    e3 |-> e4,
    e4 |-> e4
}
```

END

Example: implementing a state machine

MACHINE

P0

SEES CTX

VARIABLES

current_state

INVARIANT

current_state : STATES

INITIALISATION

current_state := e0

OPERATIONS

evolve =

BEGIN

current_state :: next[{current_state}]

END

END

Variables

- Current state of the state machine

Variables

- Next current state should belong to next definition

Example: implementing a state machine

```
IMPLEMENTATION P0_i
REFINES P0

SEES CTX

CONCRETE_VARIABLES
6/6   current_state
INITIALISATION
    current_state := e0

OPERATIONS
    evolve =
    BEGIN
6/6      CASE current_state OF
6/6        EITHER e0 THEN current_state := e1
1/1        OR e1 THEN current_state := e2
1/1        OR e2 THEN current_state := e3
1/1        OR e3 THEN current_state := e3
        OR e4 THEN skip
        END
    END
END
END
```

P02 (OK|OK|7|0|100%)
Classical view

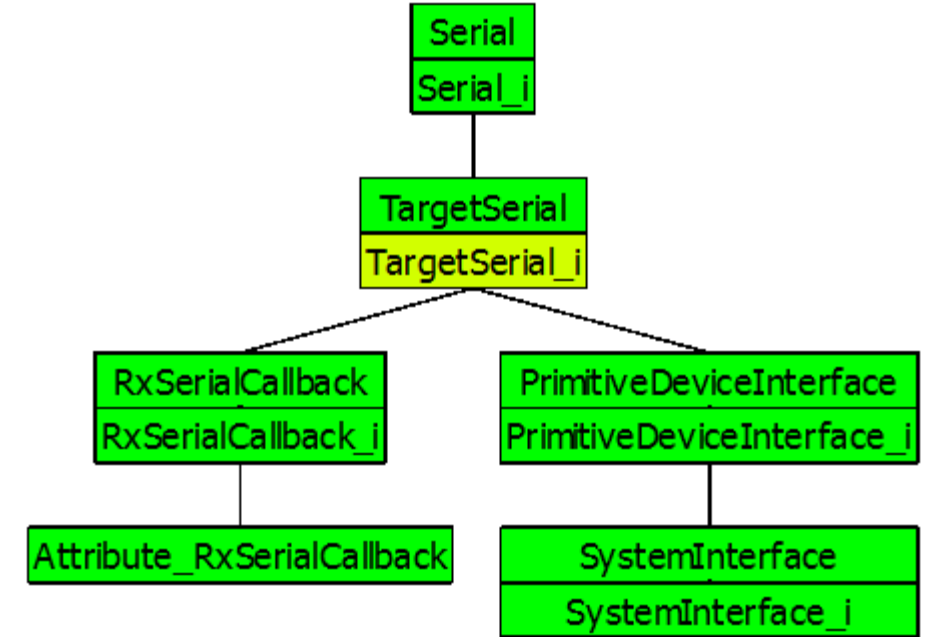
Component	TypeChecked	POs Generated	Proof Obligations	Proved	Unproved	B0 Checked
CTX	OK	OK	0	0	0	-
P0	OK	OK	1	1	0	-
P0 i	OK	OK	6	6	0	-

Operations

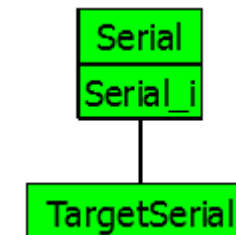
- An example of a correct implementation
- Proof is semi-automatic though

- Architecture

- Based on modules (**MACHINE + IMPLEMENTATION**)
- An operation can't call another operation of the same module
- Hence operations have to call other operations from imported components
- Some components have no implementation in B (developed manually)
- Proving an **IMPLEMENTATION** requires:
 - Its specification **MACHINE**
 - The specification of the **MACHINES** it **IMPORTS**
 - The specification of the **MACHINES** it **SEES**



Tree-like structure of a B project
(Import links)



To prove Serial_i, TargetSerial_i is not required as it complies with TargetSerial

In a B project, only implementations are translated

- Implementation have to be implementable
- No more non-determinism, sets, functions, etc.
- Only implementable types (BOOL, INT, tables of BOOL and INT)
- Only imperative constructs: valuation, IF THEN ELSE, CASE, WHILE, sequence, operation call.

Translated code is very close to original B implementation

Specification can be translated to obtain code skeleton

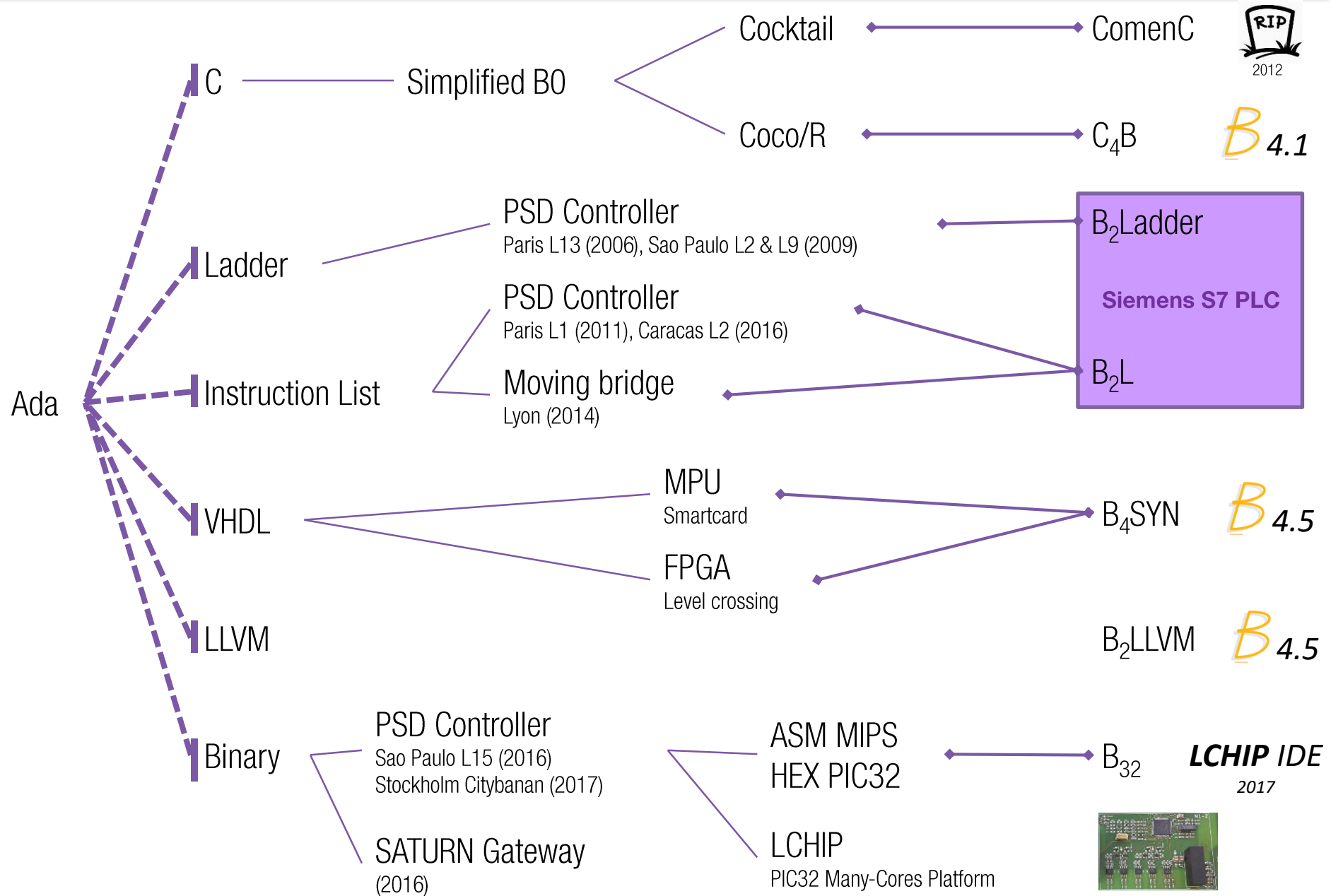
when implementation is

- Manual
- Third-party

B software can:

- be merged / integrated into “traditional” development
- reuse OSS libraries

Generating Code for Safety Critical Applications



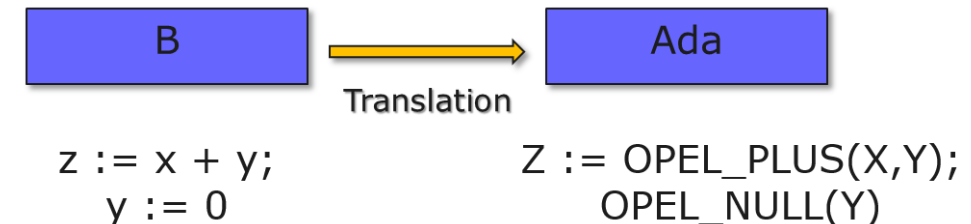
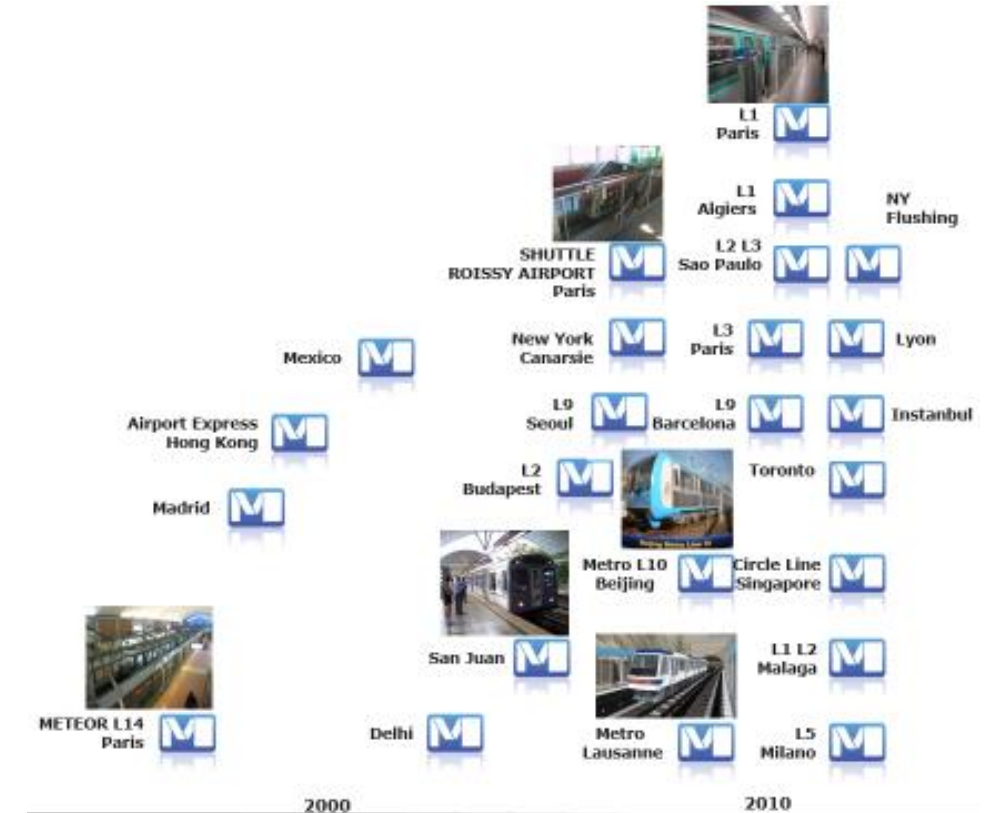
Ada

≡ Long sequence of implementations

- Paris L14 (1998) : 80 kloc
- Paris Roissy Airport Shuttle (2006): 186 kloc onboard, 50 kloc trackside
- Lille (2015): 300 kloc xml compiler

≡ Using different technologies (redundancy)

- encoding (FIDARE): 2 instances
 - inputs, outputs, variables, instructions are encoded
 - the coding key is different for each instance
- diversity (inverse mirror): 2 instances
 - one instance uses big endian model,
 - the other one executes little endian model
- specific hardware (coded secure processor): 1 instance
 - Variables have two fields: value and code. Instructions (OPELS) modified both accordingly. Mismatching value and code indicates a memory corruption.
 - Compensation tables contains tags for all possible path (masks). An unexpected value indicates a program counter corruption



C

≡ Privileged code generation path

- Code generator that produces:
 - Readable code
 - Compliant with (railways) standards: limited use of pointers
 - Restrictions on accepted BO (no multi instance)

≡ Modes

- Code generation for a:
 - Machine: skeleton for the component
 - Implementation: code for the component
 - Project: makefile and code for the components
- Profiles:
 - Code generation is highly dependant on the target
 - Predefined translation profiles allow adapt generated code

≡ INT

- INT can be redefined at project level to adapt to algorithm precision

Translator parameters

Language C

Translation profile : C9X

- C9X
- PROJECT
- LIGHT
- 01
- PIC

 New project

Bdp

Project Database TS_B\ATB431\test\bdp Browse...

Translations directory TS_B\ATB431\test\lang Browse... ☒ depends from bdp location

Project configuration

Text Encoding UTF-8

Definition of INT -2147483647..2147483647 (Legacy)

- 2147483647..2147483647 (Legacy)
- 32768..32767 (16 bits)
- 2147483648..2147483647 (32 bits)
- 9223372036854775808..9223372036854775807 (64 bits)

C

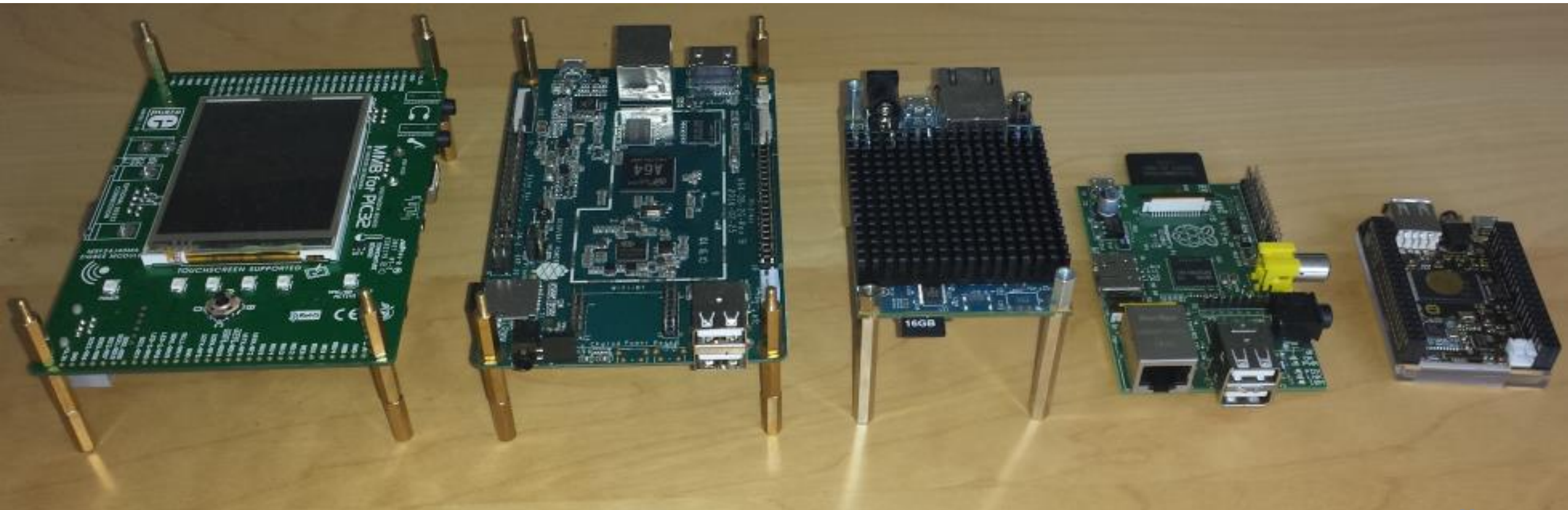
≡ Applications

- C code embedded at Sao Paulo L15 and Stockholm Citybanan



≡ Embedded systems

- Aimed at low cost hardware (from 6€ to 100€)
- Resources available in the future on github: Training Resources for the B Method



MicroChip PIC32

Pine 64

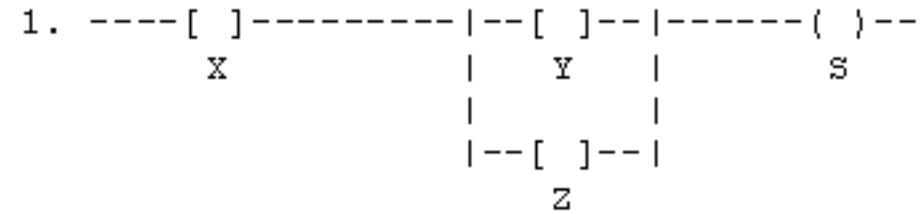
Parallella

Raspeberry Pi

²³
CHIP

≡ Programming PLCs

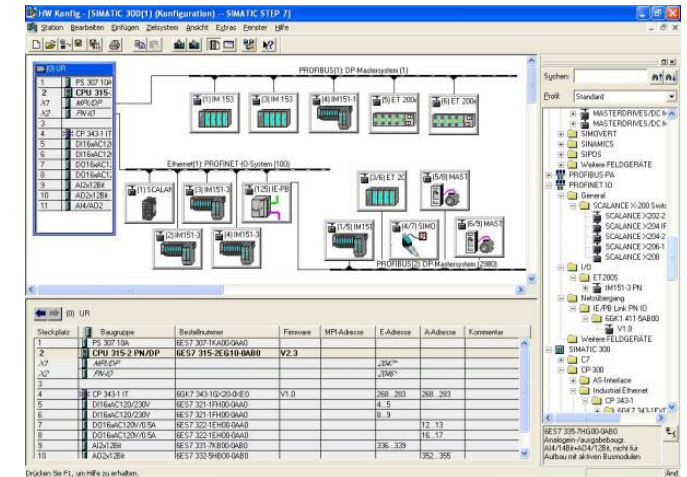
- One language (out of 5) for programming PLCs (IEC61131-3)
- Very Similar to electric circuits



The above realises the function: $S = X \text{ AND } (Y \text{ OR } Z)$

≡ Mandatory IDE

- Step7 IDE required to program Siemens-S7 to keep the SIL3 certificate
 - Ladder code generator is not connected to Step7
 - Ladder programs have to be manually entered into Step7 with the GUI
 - Cross-read required to check identity between programs
-
- Solution abandonned
 - Translator not published because too specific



Instruction List

≡ Programming PLCs

- One language (out of 5) for programming PLCs (IEC61131-3)
- Kind of assembly language for PLCs

≡ Mandatory IDE

- Step7 IDE accept Instruction List copy/paste actions
- Simpler, safer, no further cross-verification
- Translator not published because too specific

≡ Applications

- PSD controller Paris Line 1 (2015), Caracas L2 (2016)
- Bridge Crane Lyon (2013)

```
IMPLEMENTATION T003_set_i

REFINES T_empty_mch

LOCAL_OPERATIONS
  bb <-- ope_01(nn) =
  PRE
    bb : BOOL &
    nn : NAT
  THEN
    skip
  END

OPERATIONS
  bb <-- ope_01 (nn) =
  BEGIN
    IF nn=3
    THEN
      bb := FALSE
    END
  END

END
```



```
VAR_INPUT
  nn : INT;
END_VAR

VAR_OUTPUT
  bb : BOOL;
END_VAR

VAR_TEMP
END_VAR

BEGIN

NETWORK
  TITLE = RESEAU_1
  U(
    L      #nn;
    L      3;
    ==I    ;
  );
  NOT     ;
  SAVE    ;
  BEB     ;

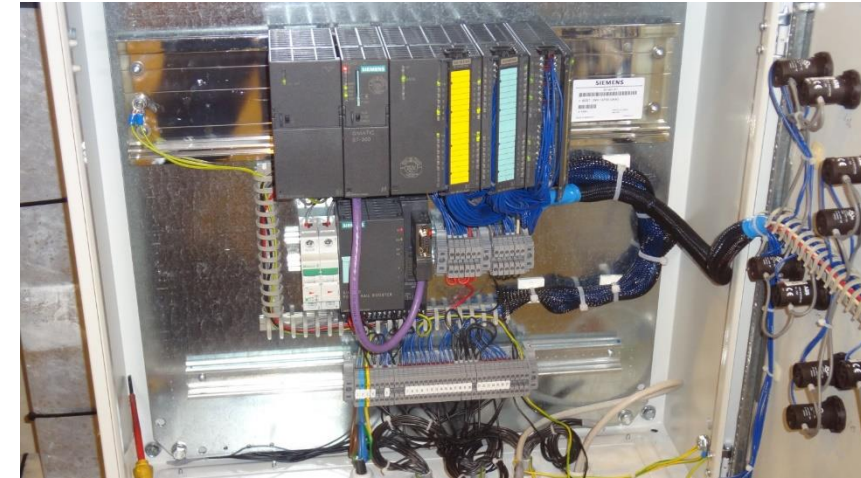
NETWORK
  TITLE = RESEAU_2
  U      "F_GLOBDB".VKE0;
  =      #bb;

END_FUNCTION
```

Instruction List

Bridge cranes (SNCF Lyon)

- Safe movements over catenary lines



Ground PLC



Crane PLC

Rationale for Using B

B is a language for proving, not for programming

Something needs to be proved

- If your requirements are only “do A_1 then A_2 then A_3 ”, do not use B

Constraints

- Operation calls of imported / sees MACHINES
- Loops should terminate
 - global while(true) can be proved or should call B operations (sequencer)

B for Safety Critical Railway Applications

≡ Reduce intervals between trains (from 120s to 90s / 75s)

- Passive security not sufficient (power off)
- Active security is required (trains have to brake when emergency)

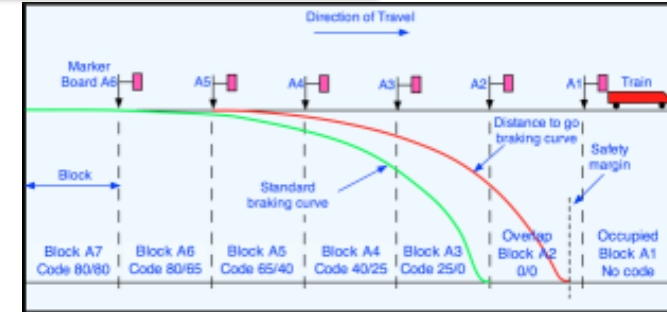
≡ SACEM ATP

- Analyzed with Z: 20 catastrophic failing scenarios found
- Massive public investment for strengthening Alstom CASE tool
- Atelier B ready for action

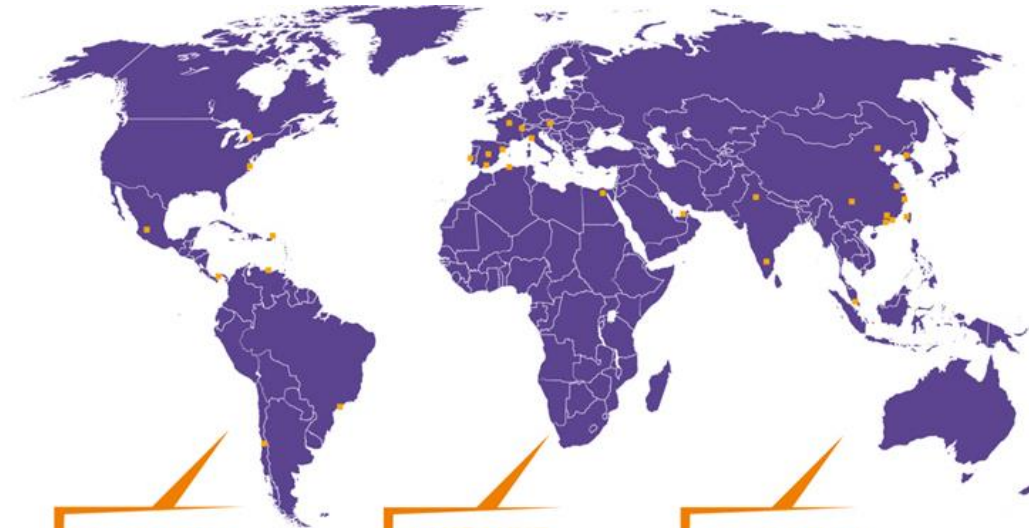
≡ B instead of program proof for

- Embedded software (Automatic Train Pilot)
- Localization: graph-based algorithms
- Energy control: integer arithmetic (braking curve)
- Emergency braking: Boolean predicates
- Trackside software (Interlocking)

≡ +30% automatic metros in the world



Braking curves



NEW YORK
SAN JUAN
MEXICO
CARACAS
SANTIAGO
SAO PAULO
PANAMA
TORONTO

LAUSANNE
MILANO
BARCELONA
MADRID
LISBON
ALGIERS
CAIRO
BUDAPEST
PARIS
MALAGA
DUBAI

BENGALORE
DELHI
SEOUL
BEIJING
SHANGAI
HONK-KONG
SINGAPOUR
NINGBO
TAICHUNG
KUNMING
SHENZHEN
GUANGHOU

Safety Critical Railway Applications

Top level implementation

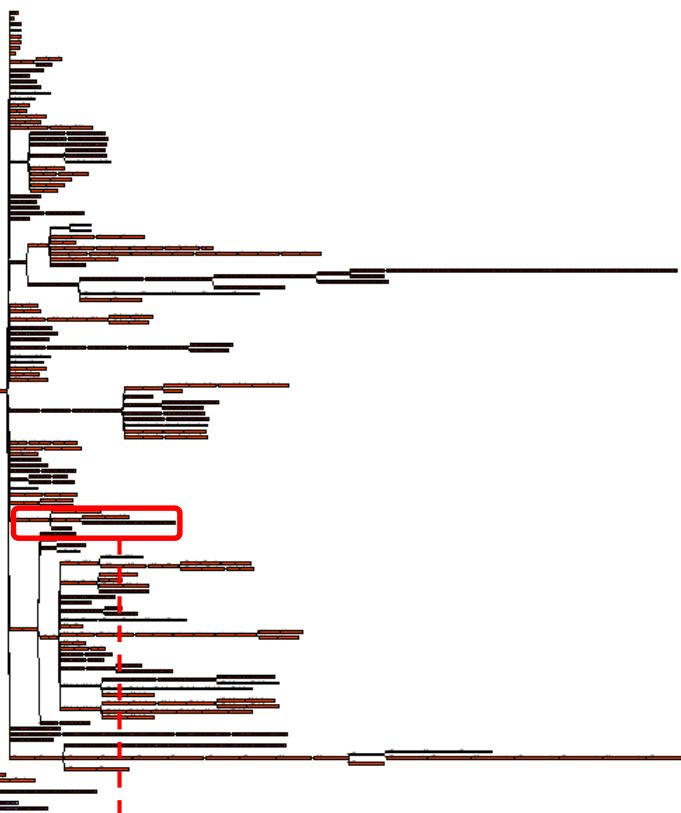
- Imports 55 components
- Specify top level one-cycle function:
 - Compute location, manage kinetic energy, control PSD, trigger emergency braking, etc.

Metrics

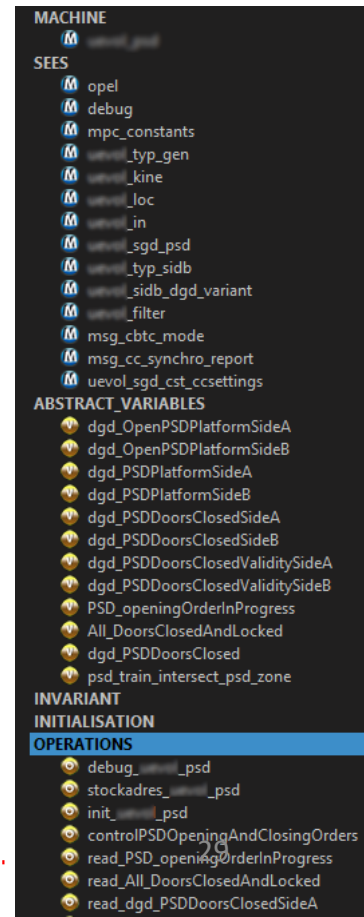
- 233 machines, 50 kloc
- 46 refinements, 6 kloc
- 213 implementations, 45 kloc
- 3 000 definitions
- 23 000 proof obligations (83 % automatic proof)
- 3 000 added user rules (85 % automatic proof)

Platform Screen Doors

- Top component: 12 variables 14 operations
- 10 components, 2 kloc specification, 2kloc implementation
- Connection with I/O through basic machines



Modern Automatic Train Protection Software (2015)



Safety Critical Railway Applications

Biggest function: Localization

where is the train ?

Post-condition of one operation:

variables become such as ...



Modern Automatic Train Protection Software (2015)

```
variables : (types &
  properties &
  properties_train &

  (loc_trainLocated = TRUE =>
    0 <= loc_locationUncertainty
    & kine_kineInvalid = FALSE
    & loc_train_track /= {}
    & first(loc_train_track) = loc_ext2Block |-> oppositeDirection(loc_ext2Dir)
    & !ii.(ii : 1..size(loc_train_track)-1 => loc_train_track(ii) : dom(sidb_nextBlock))
    & !ii.(ii : 1..size(loc_train_track)-1 => sidb_nextBlock (loc_train_track(ii)) = loc
    & #aa.(aa : 1..size(loc_train_track) & prj1(t_block,t_direction) (loc_train_track(aa))

    & loc_rearBlock = { c_cabin1 |-> loc_ext2Block, c_cabin2 |-> loc_ext1Block, c_none |->
    & (loc_rearBlock = c_block_init
      => loc_rearDir=c_up)
    & ( not (loc_rearBlock = c_block_init)
      => loc_rearDir = { c_cabin1 |-> oppositeDirection(loc_ext2Dir), c_cabin2 |-> c
    & loc_rearAbs = { c_cabin1 |-> loc_ext2Abs, c_cabin2 |-> loc_ext1Abs, c_none |-> 0
    & loc_frontBlock = { c_cabin1 |-> loc_ext1Block, c_cabin2 |-> loc_ext2Block, c_none |->
    & loc_frontDir = { c_cabin1 |-> loc_ext1Dir, c_cabin2 |-> loc_ext2Dir, c_none |-> c
    & loc_frontAbs = { c_cabin1 |-> loc_ext1Abs, c_cabin2 |-> loc_ext2Abs, c_none |-> s
  ))
```

« improved development cycle »

« Only inactive sequences can be added to the active sequences execution queue. »

Natural language
requirement

- Specification and design coding under control
- Usually more time spent during specification
- V development cycle is mainly \ development cycle

```
activation_sequence = /* Activation d'une séquence non active */
PRE ¬(sequences = sequences_actives) THEN
  ANY sequ WHERE
    sequ ∈ sequences - sequences_actives
  THEN
    sequences_actives := sequences_actives U {sequ}
  END
END;
```

B Specification

```
activation_sequence = /* Activation d'une séquence non active */
VAR sequ IN
  sequ <-- indexSequenceInactive;
  activeSequence(sequ)
END;
```

B Implementation

```
void M0_activation_sequence(void)
{
  CTX_SEQUENCES sequ;

  sequence_manager_indexSequenceInactive(&sequ);
  sequence_manager_activeSequence(sequ);
}
```

C generated code

```
0x01F970 FFFF 8B4C 2440 89C5 8D7D 0C8B 4110 89CE
0x01F980 83C6 0C8D 1485 0000 0000 8D42 0883 F807
0x01F990 7617 F7C7 0400 0000 740F 8B41 0C8D 7D10
0x01F9A0 83C6 0489 450C 8D42 04FC 89C1 C1E9 02F3
```

Binary code

- However potential problems at interfaces
 - Are requirements captured by formal specification ?
 - Is generated source code correct ?
 - Is binary code correct ?

Improved « improved development cycle »

Several techniques to speed up convergence:

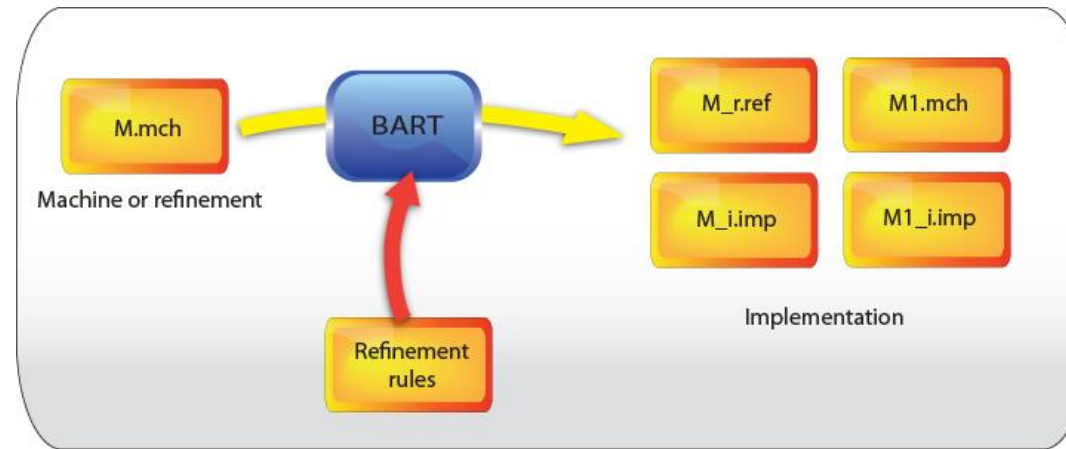
- Automatic refinement
- Traceability
- Model animation
- Simplified abstract modelling

Automatic Refinement

≡ Progressive transformation of a model containing all design decisions into a model able to be translated into an imperative language

≡ Initially invented by Matra Transport

- To support their own development methodology (abstract model first)
- In-house tools (EdithB + Bertille) to obtain an efficient process
- Refinement engine based on refinement rules

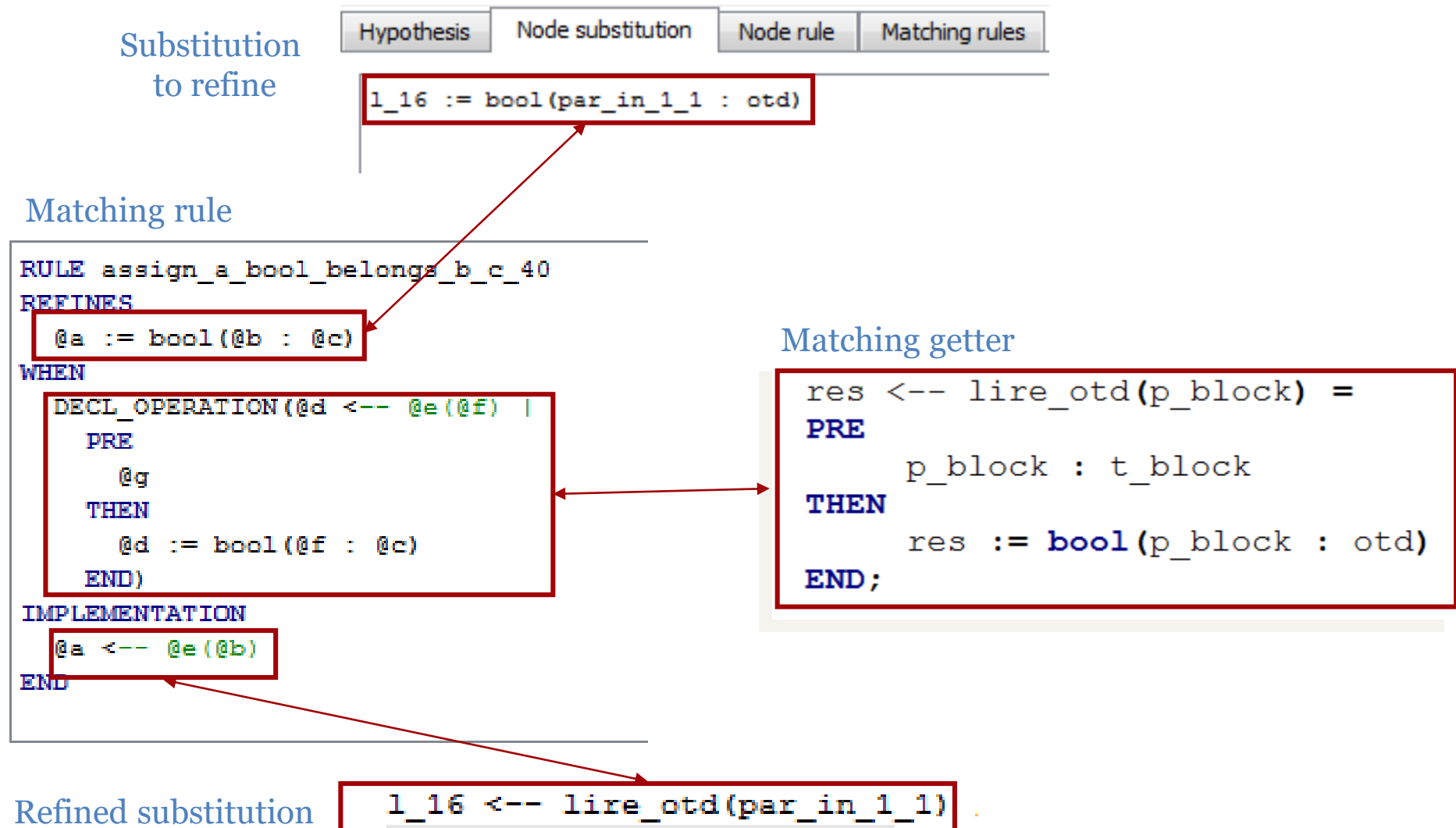


Abstract model

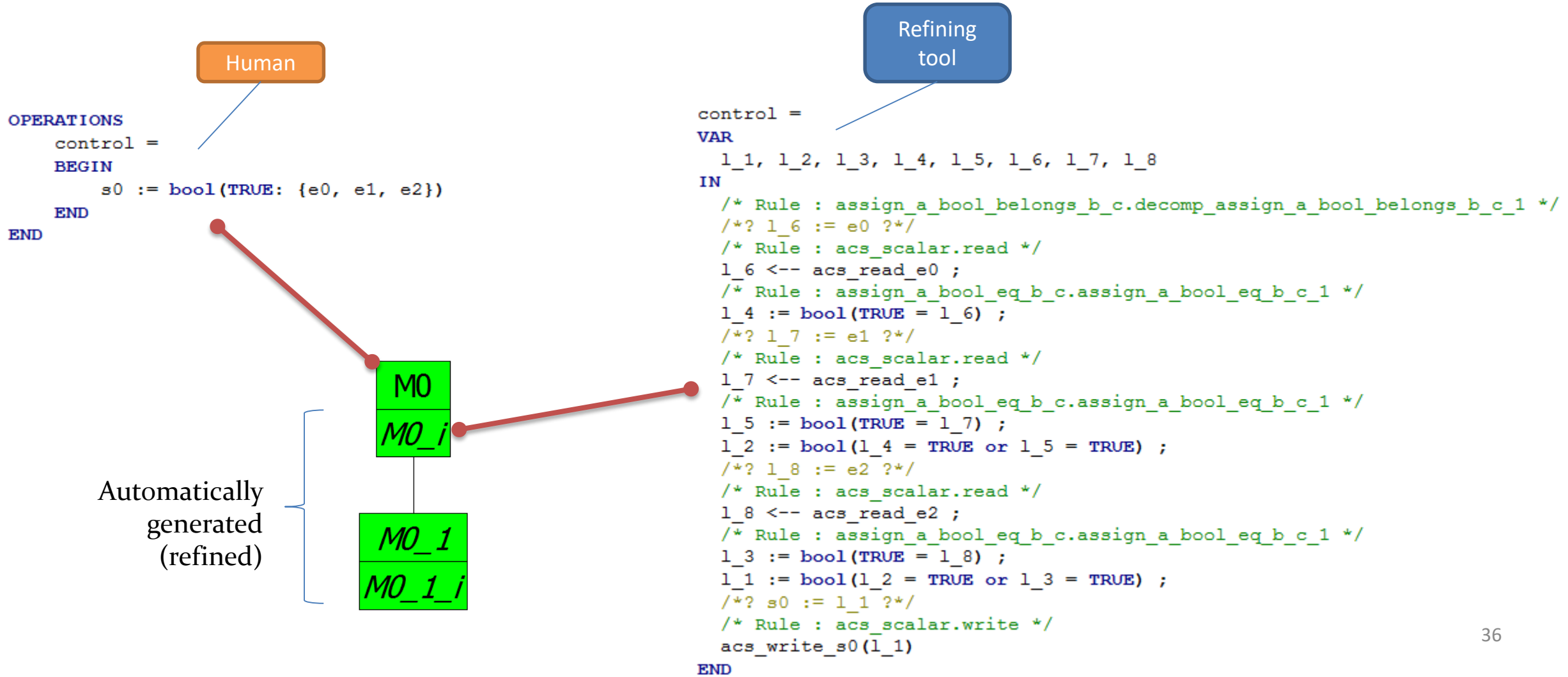
root

Modern Automatic Train Protection Software (2015)

Refinement Rules & Pattern Matching



From B Specification to B Implementation



Automatic Refinement

- ≡ No need to certify the tool
- Whatever the refinement rules, in the engine or added by users
- If the refined model is incorrect, it can't be proved

≡ Impact on projects

- Siemens claimed to divide development costs by 2 when the process is stabilized
- More little steps, more decomposition levels, more function calls, more lines of code
- Refinement patterns ease deployment of proof strategies

≡ Industrial applications

[2004] Canarsie line (ATP):

[2007] Roissy airport shuttle (ATP+ZC):

[2007] Symbolic Calculus Engine:

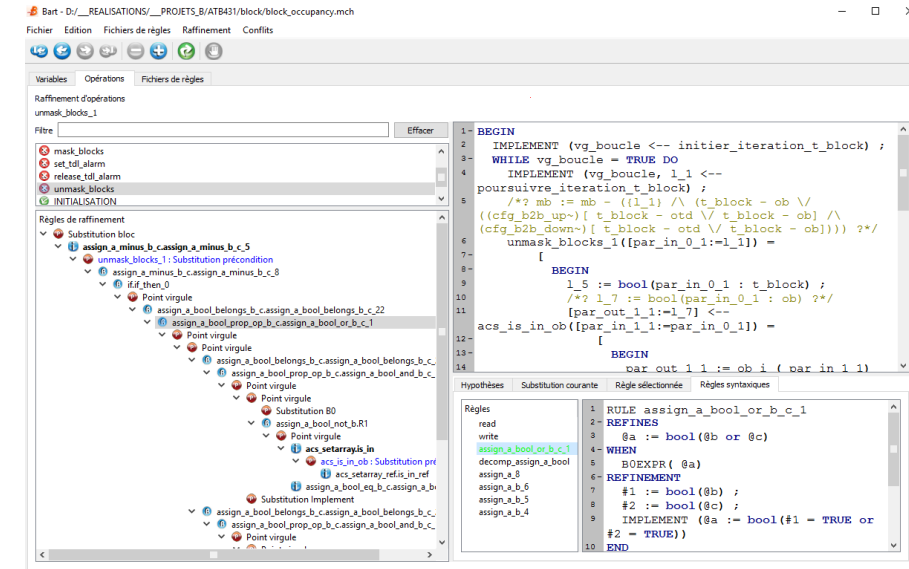
[2015] SysML Compiler :

*38k lines of handwritten B,
115k lines of generated B*

*40k lines of handwritten B
225k lines of generated B*

200k lines of generated B

300k lines of generated B



B Automatic Refinement Tool (BART) added to Atelier B in 2007,
in accordance with Siemens T.S.

Traceability / Conformance

Dedicated tool, integrated to Atelier B, to ensure traceability between natural language document and B models, over versions.

The screenshot displays the Atelier B IDE interface for the project 'uevol_ctrl_minabis_1*'. The main editor shows a B model with several requirements and assignments. A 'Requirement CC_ATP-A401464-SwRS-1292-1' dialog is open, showing the original document text and the corresponding B model body. The 'Specifications' panel on the right lists 10.2's Requirements (10.2.8) with their conformance percentages.

Requirement CC_ATP-A401464-SwRS-1292-1

Original document : Document.doc

Body :

```
Other CC Synchro Report Minabis Distance Counter for end 1(Cycle)
| Value on condition
| Case
| | 0
| | Initialization
```

Comment :

ctrl_MINABIS_counter_end1 = Other CC Synchro Report Minabis Distance Counter for end 1 (Cycle)

Specifications

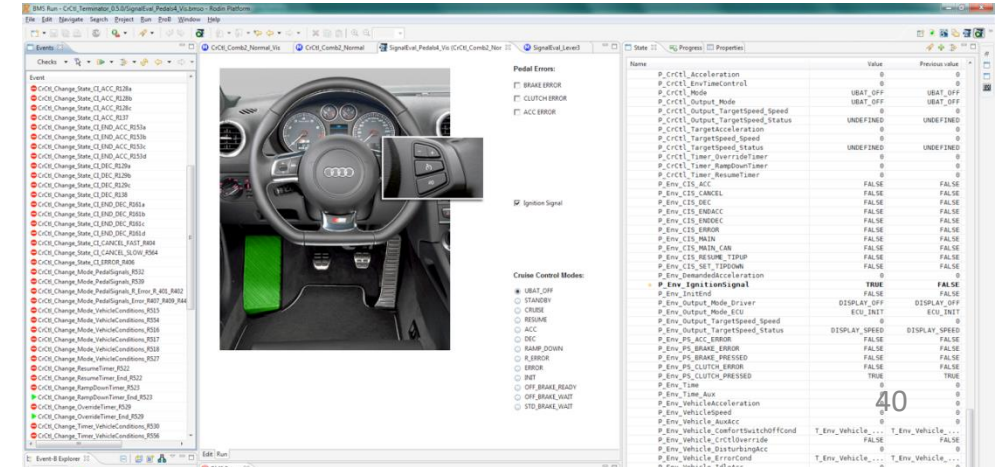
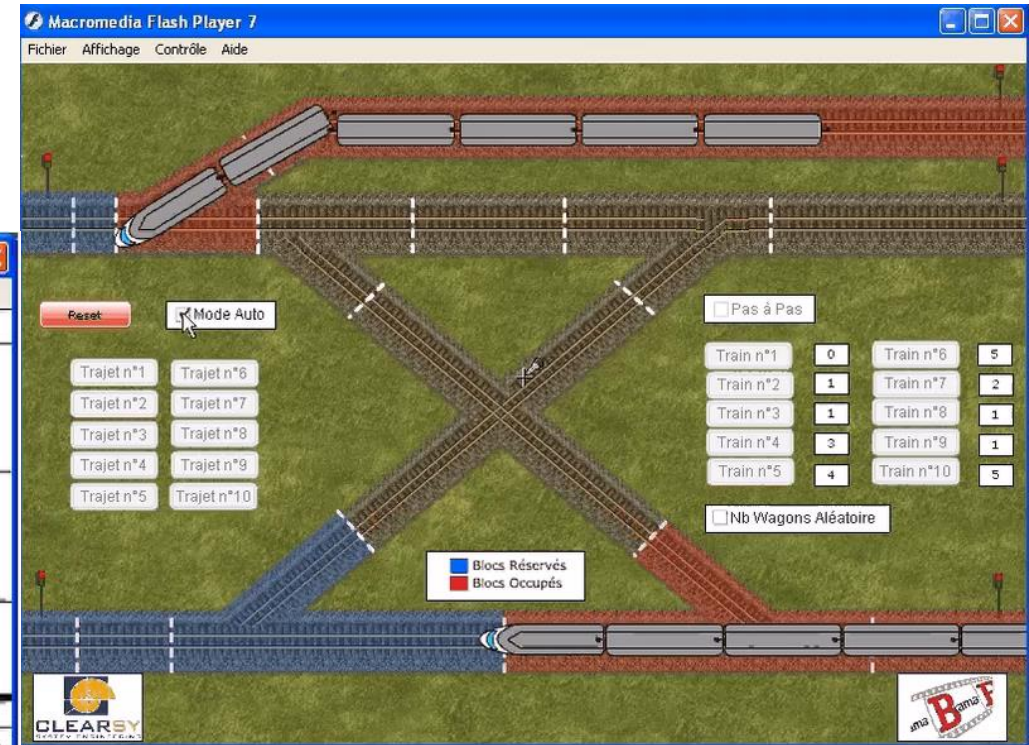
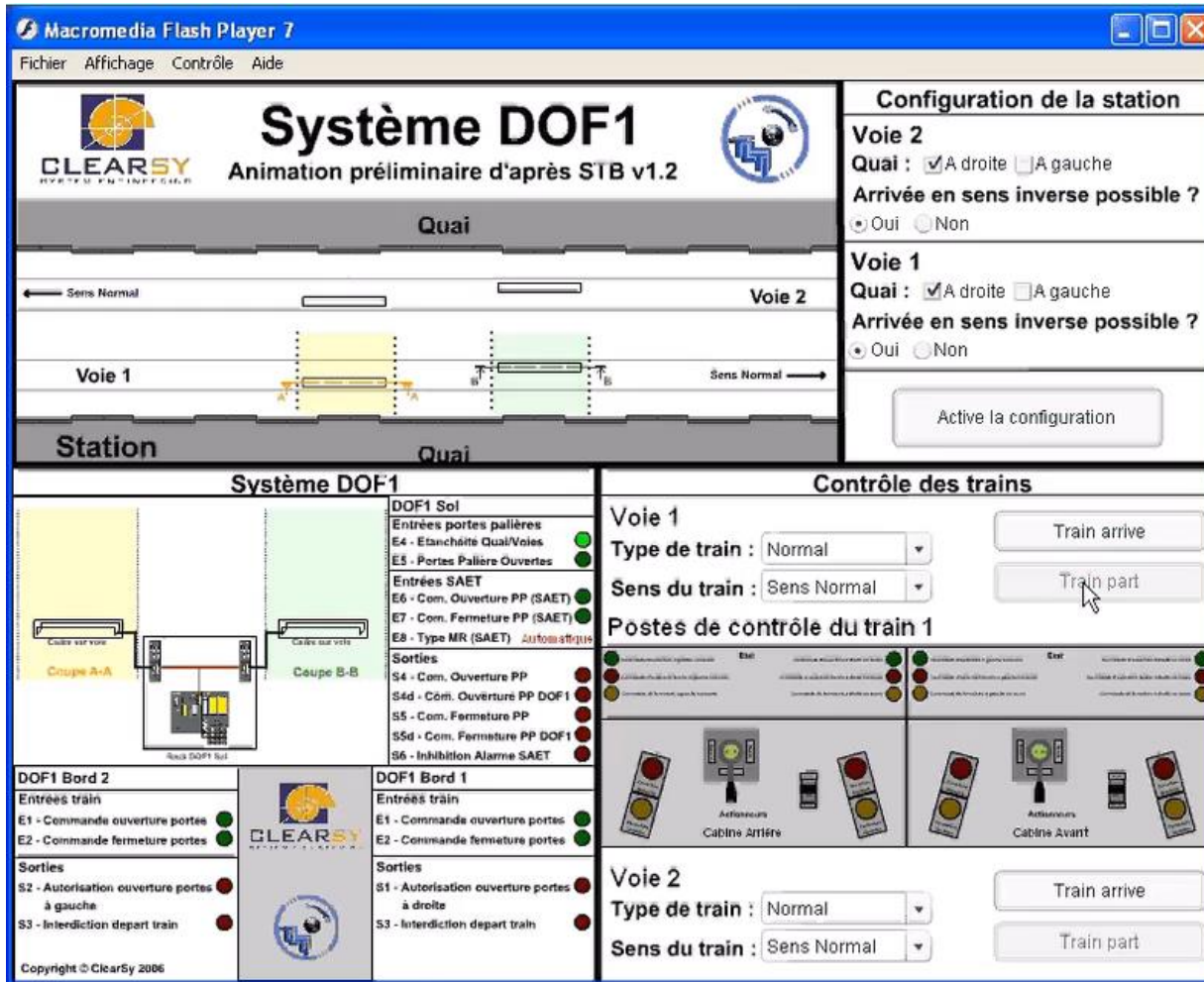
Specification	%
10.2's Requirements (10.2.8)	95.4225%
CC_ATP-A401464-SwRS-1293	100%
CC_ATP-A401464-SwRS-1292	100%
CC_ATP-A401464-SwRS-1292-2	100%
CC_ATP-A401464-SwRS-1292-1	100%
CC_ATP-A401464-SwRS-1291	100%
CC_ATP-A401464-SwRS-1290	100%
CC_ATP-A401464-SwRS-1289	100%
CC_ATP-A401464-SwRS-1271	100%
CC_ATP-A401464-SwRS-1270	100%
CC_ATP-A401464-SwRS-1269	100%
CC_ATP-A401464-SwRS-1268	100%
CC_ATP-A401464-SwRS-1267	100%
CC_ATP-A401464-SwRS-1266	100%
CC_ATP-A401464-SwRS-1265	100%
CC_ATP-A401464-SwRS-1264	100%
CC_ATP-A401464-SwRS-1263	100%
CC_ATP-A401464-SwRS-1262	0%
CC_ATP-A401464-SwRS-1261	100%
CC_ATP-A401464-SwRS-1258	100%
CC_ATP-A401464-SwRS-1257	66.6667%
CC_ATP-A401464-SwRS-1256	100%
CC_ATP-A401464-SwRS-1255	100%
CC_ATP-A401464-SwRS-1254	100%
CC_ATP-A401464-SwRS-1253	100%
CC_ATP-A401464-SwRS-1251	100%
CC_ATP-A401464-SwRS-1250	100%
CC_ATP-A401464-SwRS-1249	100%
CC_ATP-A401464-SwRS-1248	100%
CC_ATP-A401464-SwRS-1247	100%
CC_ATP-A401464-SwRS-1246	100%
CC_ATP-A401464-SwRS-1245	100%
CC_ATP-A401464-SwRS-1244	100%
CC_ATP-A401464-SwRS-1241	100%

On-going action with LACL to provide a formal framework for requirements modelling and verification, « à la KAOS »

Model Animation

Model animation for:

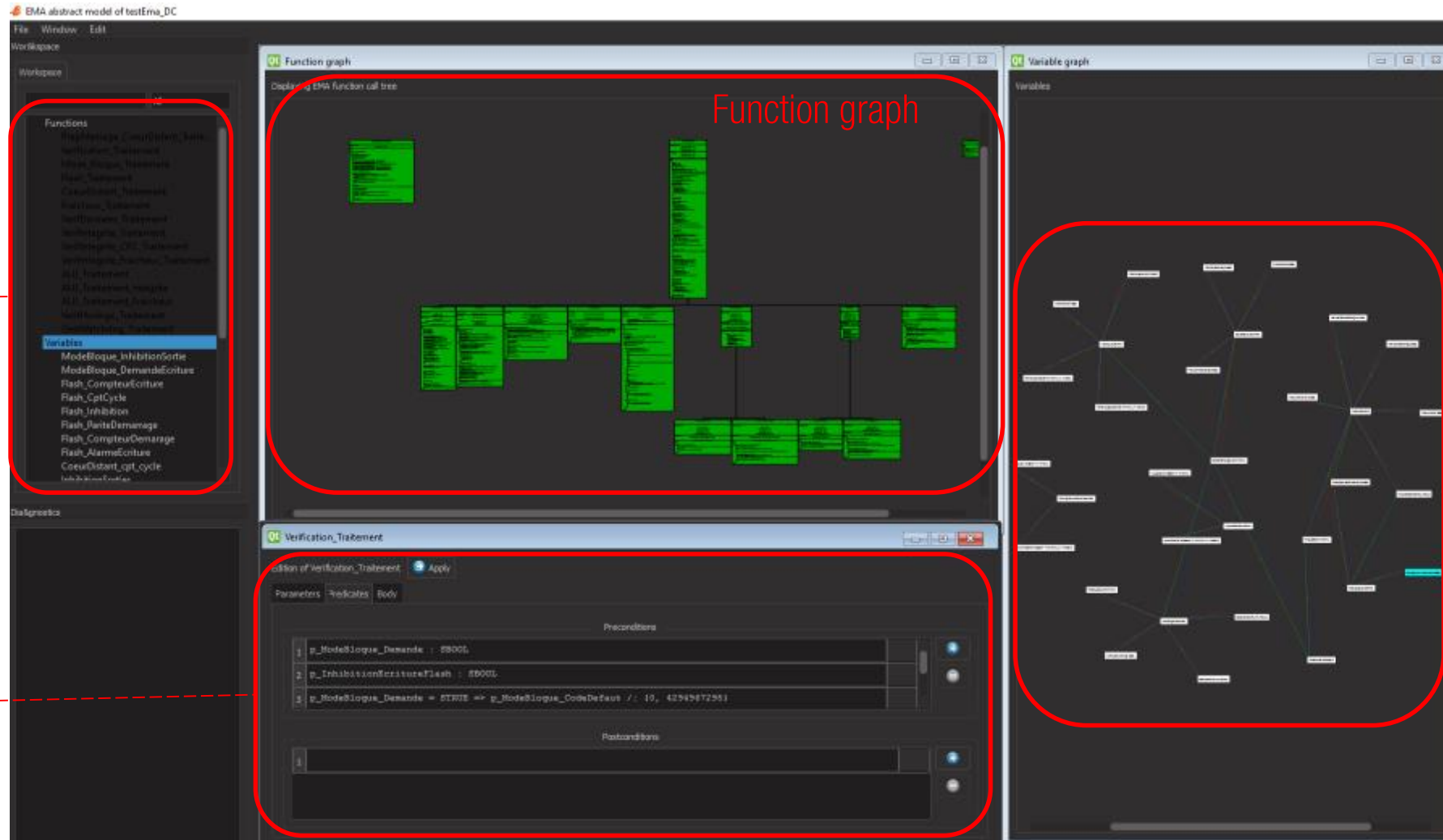
- Debugging
- Convincing experts/customers



Simplified Abstract Modelling

≡ Propose a new way of modelling

- User is incited to think in terms of functions
- Model creation, decomposition, variables allocation is left to the computer



Outline

- Functions
- Variables
- Constants

Function

- Parameters
- Predicates
- Body

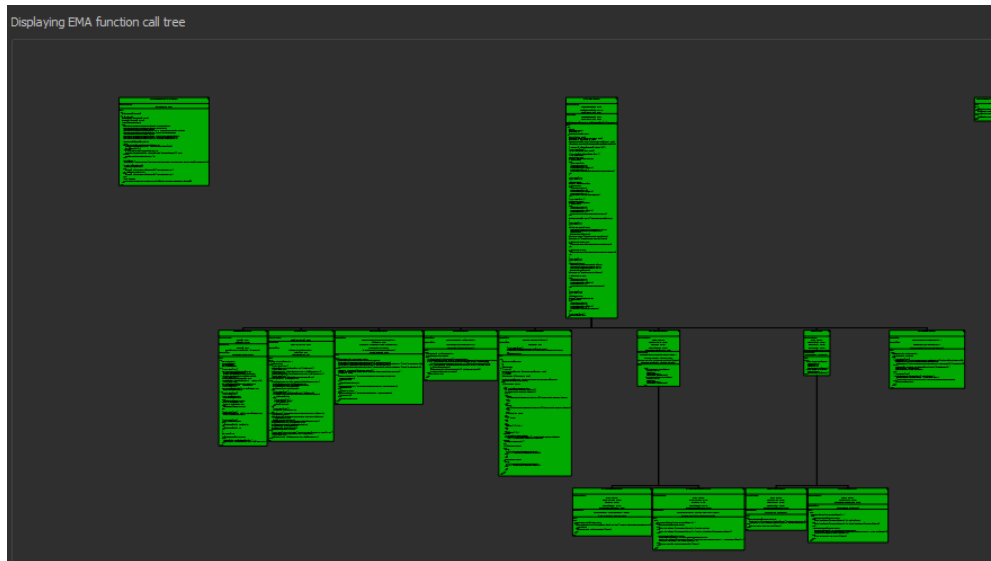
Variables graph

- What is produced
- What is consumed

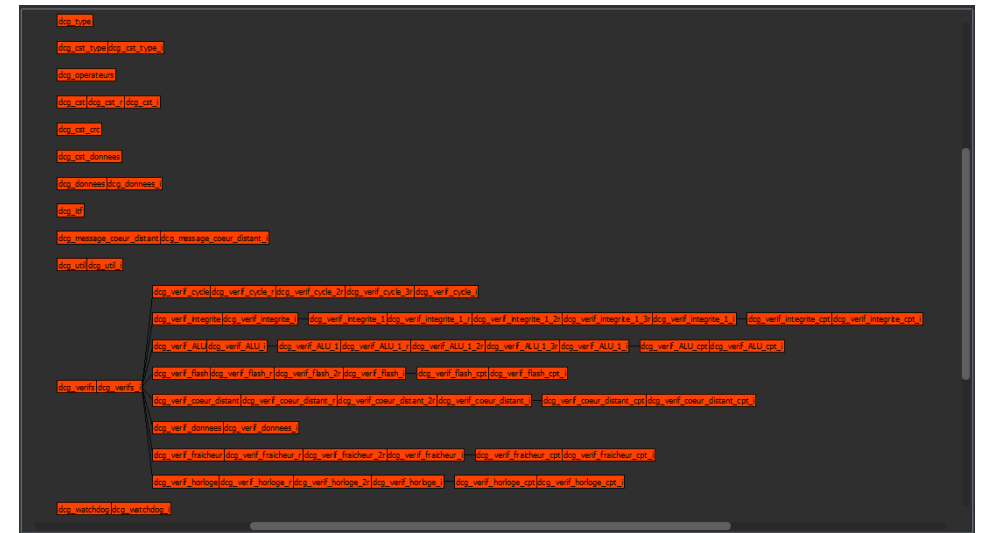
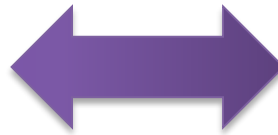
Simplified Abstract Modelling

≡ Two Modelling Views for the Same Project

- Projects can be edited whatever the view
- Providing a clear distinction between abstract model and concrete model
- Corollary: not all projects can be analysed
- Direction continuation of the Automatic Refinement approach



Abstract Modelling Editor (EMA) to appear in Atelier B 4.4



Atelier B current component view

Low Cost High Integrity Platform

Genesis

- **Safety critical applications development**

- Higher development costs (safety case)
- Expensive execution platforms
- Operational constraints

- **Impact**

- Rigid solutions
- Prohibitive costs
- Lack of available of key personal resources

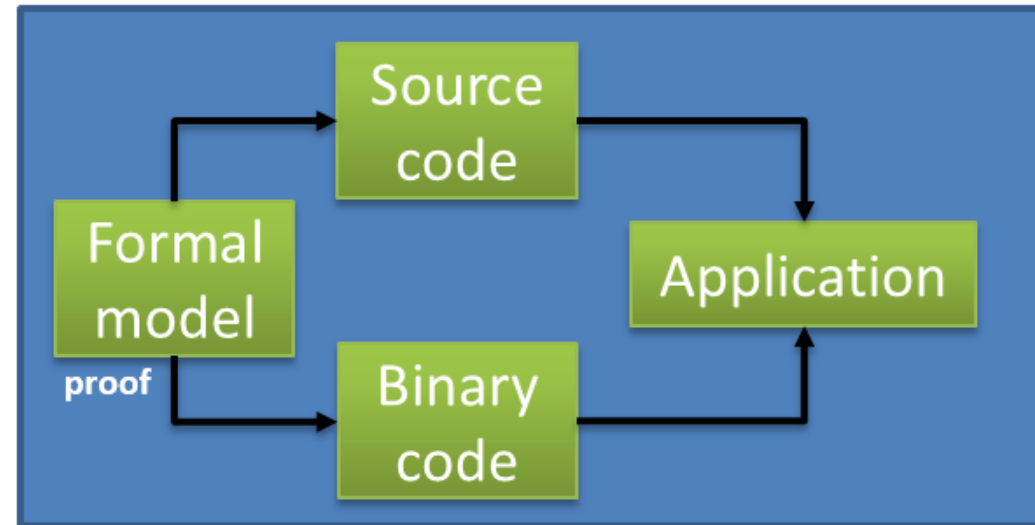


- **Secure architecture**

- Combination of software and hardware techniques compliant with EN5012{6, 8, 9}
 - Bi-processor using standard hardware
 - Redudant and proved software development process
- Design of safe (SIL4), low cost automation (+ certification kit)

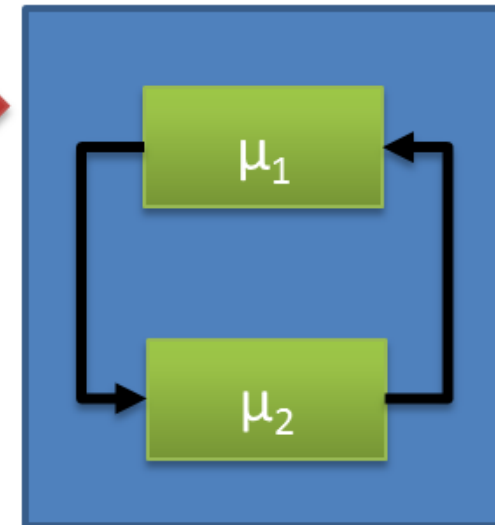
Principles (existing)

Formal software development toolchain



deployment
secure
bootloading

LCHIP execution



detection of divergent
behaviour

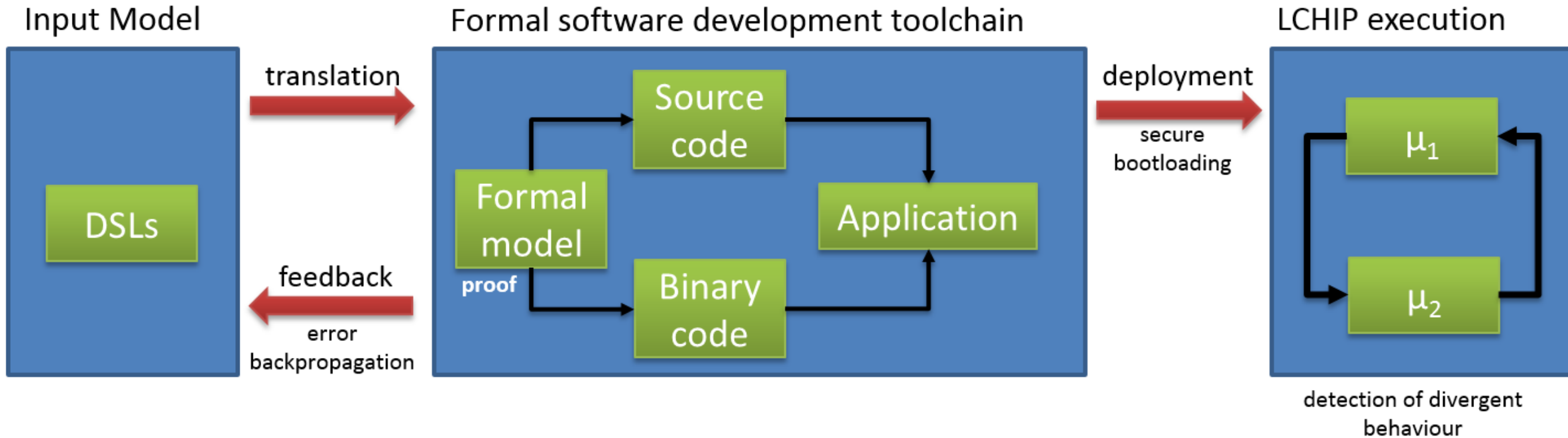
Atelier B
C code generator
Hex compiler
Linker

Unique determinist B model
C code generation and compilation
Hex binary code generation
Production of a unique binary file
Verification of correct loading

Execution on 2x PIC32
100 MIPS usable
RM if default
Instructions testing

PSD Sao Paulo et Stockholm, gateway SATURN
SIL4 Certification (CERTIFER)

Principles (LCHIP)



Translation DSLs to B
Errors backpropagated to DSLs

Unique determinist B model
C code generation and compilation
Hex binary code generation
Production of a unique binary file
Verification of correct loading
Automatic refinement of B models
Automatic proof of B models

Execution on 2x PIC32
100 MIPS usable
RM if default
Instructions testing
Daughter board
SCADA (supervision)

Low Cost High Integrity Platform

≡ 1 B model (application)

- Data
 - Integer arithmetic
 - Boolean equations
 - State machine
 - Variables typing and properties
- Cyclic software, no interrupt modifying state variables

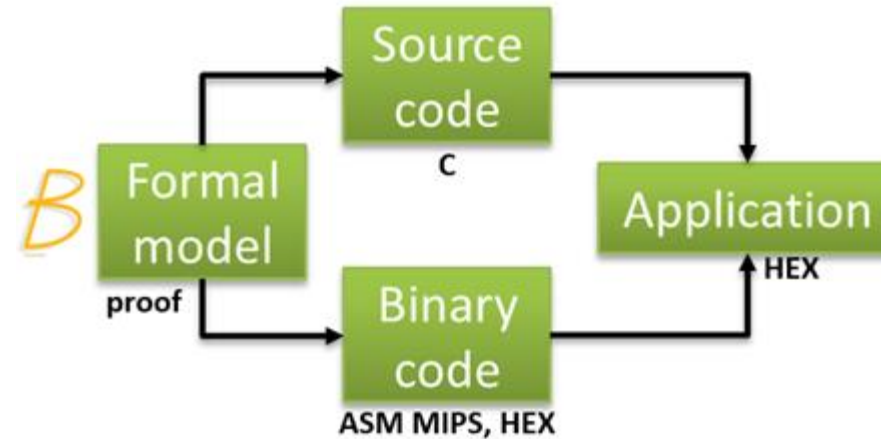
≡ Application proved vs requirements

- Implementation doesn't contradict specification
- A minima, no programming error:
 - overflow,
 - div 0,
 - tables

≡ Safety related features

- Instruction testing: instructions are tested at each iteration (fully functional microprocessor)
- Performances are checked regularly (counters) in case one microprocessor is running too slow
- Variables locations checked when loading applications (distinct variables, enough memory)

≡ 1 C code generator + commercial compiler => Hex



≡ 1 ASM MIPS / Hex code generator

≡ Low level software ensuring safety developped in B

- Once for all
- Safety features out of reach of the developers
- No requirement to certify the tools

Binary

B model

```

16 operation| =
17 BEGIN
18   IF xx = FALSE THEN
19     IF yy = FALSE THEN
20       IF st = NOMINAL THEN
21         st := DEFAULT
22       END
23     END
24   END
25 END
26 END

```

Mapping

```

= 0x00018000  machine__initialisation
= 0x00018028  operation

```

C

```

31 void M0__operation(void)
32 {
33     if(M0__xx == false)
34     {
35         if(M0__yy == false)
36         {
37             if(M0__st == M0__NOMINAL)
38             {
39                 M0__st = M0__DEFAULT;
40             }
41         }
42     }
43 }

```

Library

```

1 :0200000040001f9
2 :04800000e8ffbd27b1
3 :048004001400baff6
4 :048008001000beaff7
5 :04800c00af0a003d3
6 :048010000ae8c003b7
7 :048014001000be8f0b
8 :048018001400bf8f02
9 :04801c001800bd2764
10 :048020000800e00371
11 :048024000000000058
12 :04802800e8ffbd2789
13 :04802c001400bfafce
14 :048030001000beafcf
15 :04803400af0a003ab
16 :048038000000a3cfe
17 :04803c001004a35c0
18 :048040000000b3cf5
19 :0480440001006b3597

```

```

20 :0480480026484b017a
21 :04804c00100292dd9
22 :048050003002011f8
23 :048054000000000028
24 :048058000b00001009
25 :04805c000000000020
26 :048060000000a3cd6
27 :0480640001004a3598
28 :048068000000b3ccd
29 :04806c0001006b356f
30 :0480700026484b0152

```

```

31 :048074000100292db1
32 :048078003002011d0
33 :04807c000000000000
34 :0480800001000010eb
35 :0480840000000000f8
36 :048088000ae8c0033f
37 :04808c001000be8f93
38 :048090001400bf8f8a
39 :048094001800bd27ec
40 :048098000800e003f9
41 :04809c0000000000e0
42 :00000001ff

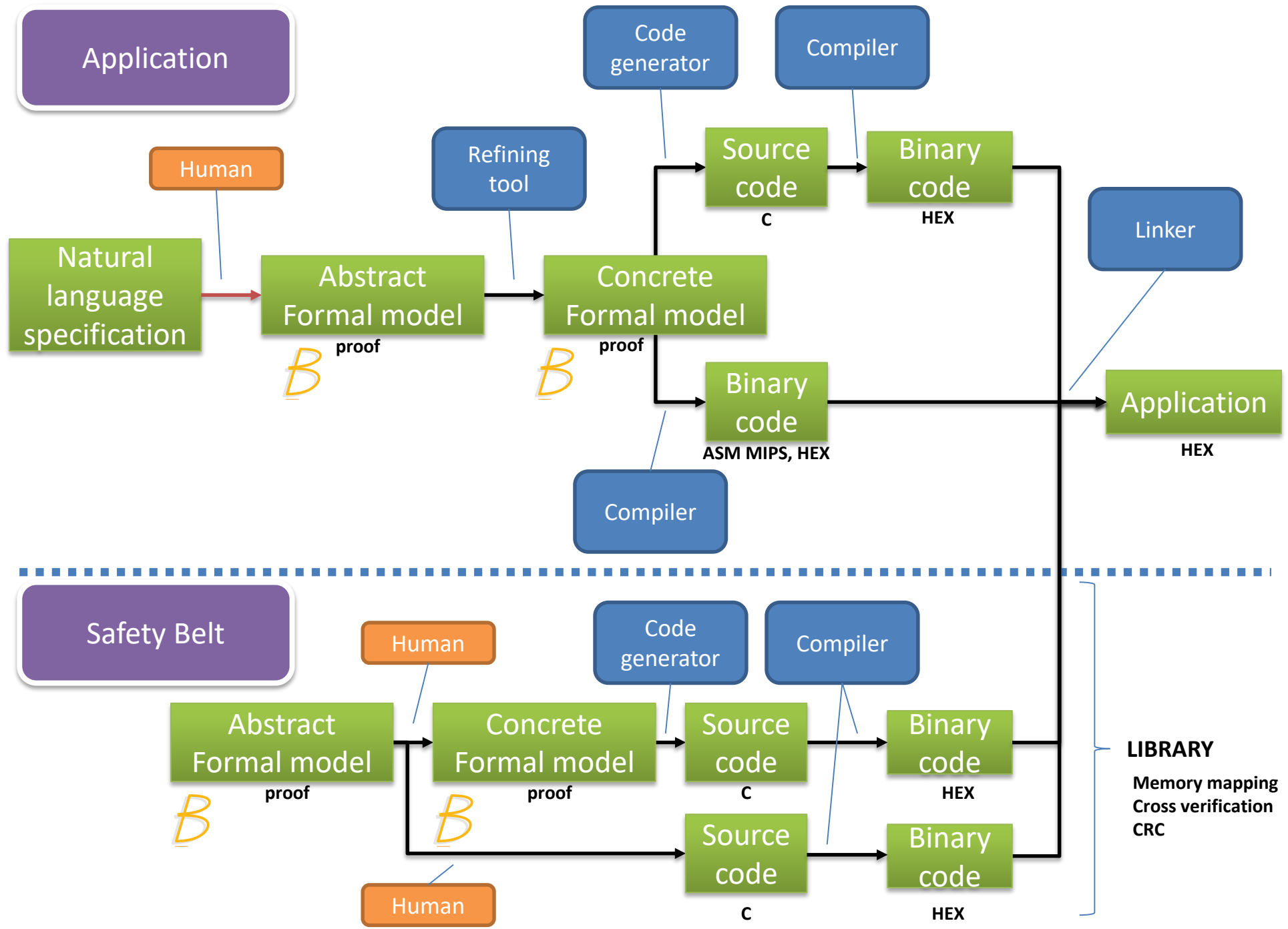
```

Hex

```

1 :0200000040001f9
2 :04800000e8ffbd27b1
3 :048004001400baff6
4 :048008001000beaff7
5 :04800c00af0a003d3
6 :048010000ae8c003b7
7 :048014001000be8f0b
8 :048018001400bf8f02
9 :04801c001800bd2764
10 :048020000800e00371
11 :048024000000000058
12 :04802800e8ffbd2789
13 :04802c001400bfafce
14 :048030001000beafcf
15 :04803400af0a003ab
16 :048038000000a3cfe
17 :04803c001004a35c0
18 :048040000000b3cf5
19 :0480440001006b3597
20 :0480480026484b017a
21 :04804c00100292dd9
22 :048050003002011f8
23 :048054000000000028
24 :048058000b00001009
25 :04805c000000000020
26 :048060000000a3cd6
27 :0480640001004a3598
28 :048068000000b3ccd
29 :04806c0001006b356f
30 :0480700026484b0152
31 :048074000100292db1
32 :048078003002011d0
33 :04807c000000000000
34 :0480800001000010eb
35 :0480840000000000f8
36 :048088000ae8c0033f
37 :04808c001000be8f93
38 :048090001400bf8f8a
39 :048094001800bd27ec
40 :048098000800e003f9
41 :04809c0000000000e0
42 :00000001ff
43

```



Low Cost High Integrity Platform

≡ Starter-kit @ Q3 2017

- Mother board (I/Os, maintenance processor, network sockets)
- Daughter card, embedding the double-processor
- IDE v1, based on Atelier B

≡ IDE v2 @ Q2 2018

≡ IDE v3 @ Q2 2019



Conclusion

≡ B is an efficient way of handling safety critical software when properties are identified

- Problems are detected while software is modelled
- Not when software is complete
 - testing,
 - program proof with assertions (Frama-C)

≡ B doesn't require PhD-only teams

- But one guru and engineers able to abstract « things »

≡ Innovations in

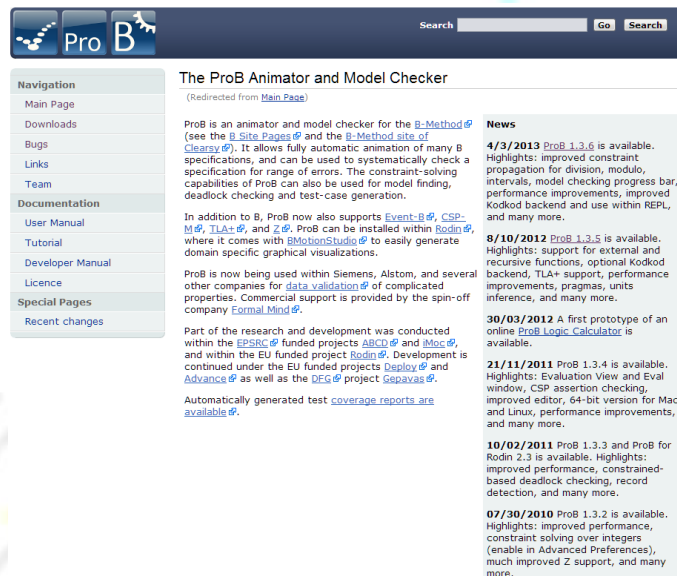
- Proof -> mostly automatic on « not too complicated » predicates (Alt-Ergo, ProB, etc.)
- Safety hardware based on low cost computers (hardware costs divided by an order of magnitude)

make B eligible for other application domains (energy/nuclear in particular)

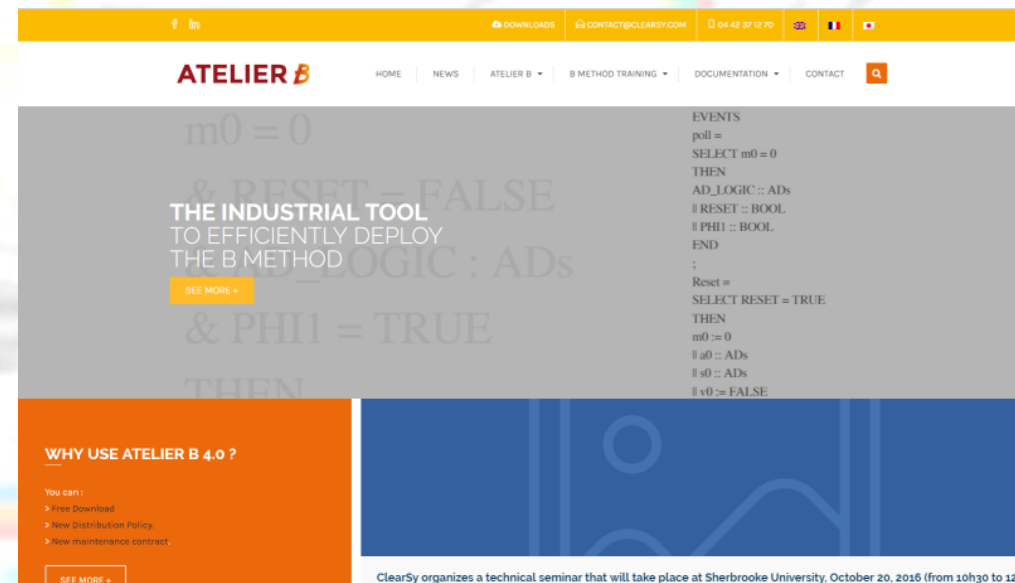
≡ Event-B

- System level models and proofs
- Smartcard certification
- Safety assessment of large systems (metro lines, military vehicles)

Thank you for your attention



<http://www.stups.uni-duesseldorf.de/ProB>



www.atelierb.eu

Thierry Lecomte
October 20, 2016
Sherbrooke